Co-Evolving Decision Agents and Agent-Managed Skill Banks for Long-Horizon Game Play

GitHub Repository

Overview

This repository provides a framework for enhancing agentic decision-making in multi-player, long-horizon games through unsupervised and synthetic experience generation. The framework integrates with multiple game environments and supports both training-free (RAG-based) and trainable (RL-based) agent architectures. This readme outlines each module, gives initial instructions for vibe coding, and aims to ease integration and debugging (including function definitions and ToDo lists for each class).

Contents: 1. Environments · 2. Data structure · 3. Skill agent · 4. Decision-making agent · 5. Experience synthesis · 6. Trainer code · ToDo (unfinished)

Quick Links

- 🔗 **Repository**: GitHub - Game-AI-Agent
- 📦 **Environment Wrappers** — env_wrappers/: NL wrappers and evaluation for game environments
- Overcooked AI — overcooked_ai; eval: evaluate_overcooked/
- Avalon · Diplomacy — AgentEvolver Games; eval: evaluation_evolver/
- GamingAgent — LMGame-Bench (2048, Sokoban, Tetris); GamingAgent; eval: evaluate_gamingagent/
- VideoGameBench — DOS games; eval: evaluate_videogamebench/
- 🔍 **RAG & Embeddings** — rag/: Text (default Qwen3-Embedding-0.6B), multimodal (default Qwen3-VL-Embedding-2B); config: `RAG_EMBEDDING_MODEL`, `MULTIMODAL_EMBEDDING_MODEL`. rag/README.md
- 🎮 **Decision Agent** — decision_agents/: VLM step-by-step play with a **two-turn micro-loop** per timestep: `take_action` (primitives or `QUERY_MEM` / `QUERY_SKILL` / `CALL_SKILL`) → `reward`. Uses a skill bank (from skill_agents) for retrieval; `QUERY_SKILL(key)` returns a micro_plan and contract.
- **Core**: agent.py — `VLMDecisionAgent`, `run_tool()`, `run_episode_vlm_agent()`; dummy_agent.py — game detection, action extraction.
- **Helpers**: agent_helper.py — `get_state_summary()`, `infer_intention()`, `EpisodicMemoryStore` (RAG-backed memory), `skill_bank_to_text()`, `query_skill_bank()`.
- **Reward**: reward_func.py — `RewardConfig`, `RewardComputer`; **r_total** = r_env + w_follow×r_follow + r_cost (query_mem_cost, query_skill_cost, call_skill_cost, skill_switch_cost).
- **Tools**: `take_action`, `reward`, `get_state_summary`, `get_intention`, `query_skill`, `query_memory`. See decision_agents/README.md.
- 📚 **Skill Agents** — skill_agents/: Build and maintain a Skill Bank from trajectories; consumed by decision_agents for `query_skill`.
- **Orchestrator** SkillBankAgent: `ingest_episodes` → segment (Stage 1+2) → learn contracts (Stage 3) → maintain bank (Stage 4) → `query_skill`. Methods: `segment_episode()`, `run_contract_learning()`, `run_bank_maintenance()`, `run_until_stable()`.

- **Stage 1** boundary_proposal/: Candidate cut points C (signals: rule-based or LLM `env_name="llm"`; optional RAG change-point; `merge_radius`). README
- **Stage 2** infer_segmentation/: Decode over C with preference scorer (LLM → Bradley–Terry) → segments + labels (bank + `__NEW__`). README
- **Stage 3** stage3_mvp/: Effects contract learn/verify/refine; NEW → pool.
- **Stage 4** bank_maintenance/: Split/merge/refine; `materialize_new_skills`. Alt: stage4_bank_update/.

- **Query & storage**: SkillQueryEngine (RAG + keyword/effect), SkillBankMVP, tool_call_reward, skill_evaluation/. skill_agents/README.md · PLAN.md

- 🏋️ **Training** — trainer/: Co-evolution of both agents.

- **Agent A (Decision)**: GRPO — primitives + `QUERY_MEM` / `QUERY_SKILL` / `CALL_SKILL`; reward = r_env + shaping + costs.
- **Agent B (SkillBank)**: Hard-EM (decode → update → gate); optional boundary/tie-breaker learners.

- Shared: metrics, reward_shaping, eval_harness. Entry: launch_trainer, launch_coevolution. trainer/README.md

- ▶️ **Inference** — inference/: Run the decision agent and store rollouts in data_structure format (`Episode + Experience`). `run_inference(env, task=..., episode_buffer=..., save_path=...)`; `rollout_to_episode(rollout, task=...)` to convert an existing rollout. inference/README.md

---

1. Environments

This framework integrates with the following game environments. Each has an NL wrapper in env_wrappers/ and a dedicated evaluation script folder.

Environment	Source	Wrapper	Evaluation
Overcooked AI	overcooked_ai — cooperative human-AI benchmark (Overcooked game)	overcooked_nl_wrapper.py	evaluate_overcooked/
Avalon	AgentEvolver Games — hidden-role deduction	avalon_nl_wrapper.py	evaluation_evolver/
Diplomacy	AgentEvolver Games — strategic negotiation	diplomacy_nl_wrapper.py	evaluation_evolver/
GamingAgent (LMGame-Bench)	GamingAgent — 2048, Sokoban, Tetris, etc.	gamingagent_nl_wrapper.py	evaluate_gamingagent/
VideoGameBench	DOS games (JS-DOS in browser;	videogamebench_dos_nl_wrapper.py	evaluate_videogamebench/

Environment	Source	Wrapper	Evaluation
	Game Boy excluded)		

Decision & Skill Agents: Co-evolution

The **decision_agents** (VLM decision-making agent) and **skill_agents** (Skill Bank pipeline) are designed to **co-evolve**: the decision agent plays the game using skills retrieved from the bank, and new trajectories are fed back into the skill pipeline to refine and extend the bank.

Big picture

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          CO-EVOLUTION LOOP                               │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│   ┌──────────────────────┐          ┌──────────────────────┐            │
│   │   decision_agents    │          │   skill_agents       │            │
│   │   VLMDecisionAgent   │          │   SkillBankAgent     │            │
│   │                      │          │                      │            │
│   │ • step(obs) → tool   │  query   │ • ingest_episodes()  │            │
│   │ • take_action        │ ──────►  │ • segment → Stage 3  │            │
│   │ • query_skill(key)   │ ◄──────  │ • maintain bank      │            │
│   │ • query_memory       │  bank    │ • query_skill(key)   │            │
│   │ • reward (r_follow)  │          │ • add/remove/update  │            │
│   └──────────────────────┘          └──────────────────────┘            │
│              │                                │                         │
│              │                                │                         │
│              │ run_episode_vlm_agent(env)     │ run_until_stable()      │
│              ▼                                ▼                         │
│   ┌──────────────────────┐          ┌──────────────────────┐            │
│   │  Episode (trajectory) │ ──────► │  Skill Bank (JSONL)   │           │
│   │  observations, actions│  feed   │  contracts, reports   │           │
│   │  rewards, done        │  back   │  versioned, queryable │           │
│   └──────────────────────┘          └──────────────────────┘            │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

With training (trainer/): the same loop is driven by **GRPO** (decision agent) and **Hard-EM** (skill bank). The orchestrator trainer/launch_coevolution.py runs GRPO continuously, periodically freezes a rollout batch, runs the SkillBank EM trainer (propose cuts → decode → contracts → update → SkillEval gating), and deploys the new bank only if fixed-seed evaluation passes. See trainer/README.md.

How they connect

Role	decision_agents	skill_agents
Provides	Game play: one action per step, optional retrieval (`query_skill`, `query_memory`)	Skill Bank: segment trajectories, learn effect contracts, split/merge/refine skills
Consumes	Skill bank (list of skills + contracts for prompt and `QUERY_SKILL`)	Raw episodes (from rollouts or demos) to segment and extract skills
Interface	`VLMDecisionAgent(skill_bank=...)`; `run_tool(TOOL_QUERY_SKILL, {"key": "..."})`	`SkillBankAgent.query_skill(key)` or `SkillQueryEngine`; `skill_bank_to_text(bank)` for prompt

The decision agent's `skill_bank` can be a **SkillBankMVP** (plain storage) or a **SkillBankAgent** (full pipeline). Helpers (`skill_bank_to_text`, `query_skill_bank`) accept both and use the richest API available.

Do they use the RAG models (rag/) for memory/skill query?

Yes. Both agents now use RAG embeddings (default: Qwen3-Embedding-0.6B) for retrieval:

Component	Memory query (`query_memory`)	Skill query (`query_skill`)	RAG (rag/) usage
decision_agents	**Yes.** EpisodicMemoryStore embeds every memory via rag/TextEmbedder and retrieves by **cosine similarity + keyword overlap** (blended, default 70% embedding / 30% keyword). The embedder is auto-loaded from `rag.get_text_embedder()`. Falls back to keyword-only if RAG is unavailable.	**Yes** (via SkillQueryEngine). Uses query_skill_bank which delegates to **SkillQueryEngine**.	Uses rag/ `TextEmbedder` for both memory and skill retrieval.
skill_agents	N/A (skill_agents do not implement memory query).	**Yes.** SkillQueryEngine embeds all skill descriptions (ID + effect literals) via RAG `TextEmbedder` and scores queries by **cosine similarity + keyword Jaccard** (blended, default 60% embedding / 40% keyword). Auto-loads `rag.get_text_embedder()`. `query_by_effects` also uses embedding scores. Falls back to keyword-only if RAG is unavailable.	Uses rag/ `TextEmbedder` for skill retrieval. Also **optional** in **boundary proposal**: skill_agents/boundary_proposal can take a RAG embedder for embedding change-point scores.

Both agents gracefully degrade to keyword-only retrieval if the RAG module or model is unavailable (import fails or embedder init fails).

Typical workflow

1. **Bootstrap (optional)**
   Load or create an initial skill bank (e.g. from a few hand-labelled or pre-segmented episodes).
   `skill_agent = SkillBankAgent(bank_path="..."); skill_agent.load()` or `skill_agent.ingest_episodes(seed_episodes)`.

2. **Play**
   Run the VLM agent with that bank:
   `run_episode_vlm_agent(env, agent=VLMDecisionAgent(skill_bank=skill_agent), ...)`.

3. **Feed back**
   Collect the episode (observations, actions, rewards) into an `Episode` and ingest: `skill_agent.ingest_episodes([episode])` then optionally `skill_agent.run_until_stable()`.

4. **Repeat**
   The bank improves (new skills from `__NEW__`, splits/merges/refinements), so the next run has better `query_skill` results and reward shaping.

Trainable co-evolution: Use trainer/ to run both agents in a training loop. The **Decision Agent** is trained with **GRPO** (group sampling + ranking on r_total); retrieval actions (`QUERY_MEM`, `QUERY_SKILL`, `CALL_SKILL`) are first-class actions with costs. The **SkillBank Agent** is updated via **Hard-EM** (decode → learn contracts → update → gate); no global LLM scoring. Shared rollout schema and reward contract live in `trainer/common/`; configs in `trainer/common/configs/decision_grpo.yaml` and `skillbank_em.yaml`. Run co-evolution: `python -m trainer.launch_coevolution --decision-config ... --skillbank-config ...`. See trainer/README.md.

See decision_agents/README.md for the VLM agent API and skill_agents/README.md for the pipeline and query usage.

How reward is computed (decision + skill agents)

Reward for the decision/skill stack is computed in two places:

- **decision_agents** (`reward_func.py`): **r_total = r_env + w_follow × r_follow + r_cost**
- **r_env**: Raw environment reward from `env.step`.
- **r_follow**: Skill-following shaping. The active skill's contract has `eff_add` predicates (e.g. `at_pot`, `onion_in_pot`). Each predicate is checked against the **current observation text** (tokenize predicate and observation; predicate counts as satisfied if all its tokens appear in the observation). The agent gets a **per-predicate bonus** for each **newly** satisfied predicate, a **completion bonus** when all are satisfied, and a **small penalty** when no new predicate is satisfied this step. This is stateful over the episode (tracks which predicates were already satisfied).

- **r_cost**: Negative costs for tool use: **query_mem_cost**, **query_skill_cost**, **call_skill_cost**, and **skill_switch_cost** when the active skill changes.

- **skill_agents** (`tool_call_reward.py`): **r_total = w_relevance × r_relevance + w_utility × r_utility** (for agentic RL: "was this tool call good?")

- **r_relevance**: For **query_skill**, the retrieval score (from `SkillQueryEngine.query(key)` or from `retrieved_result["score"]`) scaled by `relevance_scale`. For **query_memory**, an optional `retrieval_quality` in [0,1] scaled the same way.
- **r_utility**: For **query_skill** and **call_skill**, the skill's **eff_add** predicates are checked against the **outcome observation** text (after the tool call). Same token-in-text rule: each predicate satisfied in the outcome gets a per-predicate bonus; if all are satisfied, an extra completion bonus. No state across steps—purely outcome-based.

Predicate satisfaction in both modules is **text-based**: tokenize the predicate (e.g. `onion_in_pot` → tokens), then require all tokens (length ≥ 2) to appear in the observation/outcome string (case-insensitive).

---

2. Data structure

This part defines how **experiences**, **episodes**, and **sub-task (skill)** segments are represented and cleaned from external sources into a uniform format, with optional fields for intention, summary, and quality for RAG and replay.

- **Data structure details** (class definitions, buffers, serialization) are in **data_structure/**, in particular data_structure/experience.py and data_structure/helper.py.
- **Usage** — how episodes and experiences are produced and consumed:
- **decision_agents/**: produces rollout data (observations, actions, rewards, reward_details) via `run_episode_vlm_agent()`; can be converted to `Experience` / `Episode` for downstream use.
- **skill_agents/**: consumes `Episode` (list of `Experience`) for boundary proposal and segmentation; produces `SubTask_Experience` segments and skill labels. See skill_agents/infer_segmentation/episode_adapter.py, skill_agents/boundary_proposal/, and skill_agents/README.md.

Experience

Same as in standard RL: **state, action, intention, reward, next state, done**. Implemented in data_structure/experience.py as `Experience` with required fields plus optional: **intentions**, **tasks**, **sub_tasks**, **summary**, **summary_state**, **idx**, **sub_task_done**; synthetic fields **is_synthetic**, **syn_experience_quality**. Long-term goal, short-term goal, synthesis label, and experience quality score can be filled as needed. A field for experience value is left for prioritized replay. The **summary** (and **summary_state**) support RAG and embedding-based retrieval. Depending on the environment, not all fields are provided; the actual state used by LLM agents is often a **summary of the state**, which also serves as context for RAG retrieval.

Episode

An **episode** is a sequence of experiences (time-length limited) with final reward, length, and optional outcome/summary. Implemented as `Episode` in data_structure/experience.py: **experiences**, **task**, **outcome**, **summary**; methods include **get_reward()**, **get_length()**, **set_outcome()**, **separate_into_sub_episodes()** (splits by sub-task indices into `SubTask_Experience`).

Sub-task (skill) experience

SubTask_Experience represents a contiguous segment of experiences that accomplish a sub-task (strategy/skill): **sub_task**, **final_goal**, **sub_task_experience** (list of `Experience`), **outcome_experiences**, **summary**, **outcome_summary**, **length**, **cumulative_reward**. Used by skill_agents for skill labeling and the skill bank pipeline.

Buffers

- **Experience_Replay_Buffer**: FIFO buffer of experiences; **add_experience()** (single, list, or full `Episode`), **sample_experience()**, **get_experience_summary()**.
- **Episode_Buffer**: FIFO buffer of full episodes; **add_episode()**, **sample_episode()**, **get_episode_summary()**, **save_to_json()** / **load_from_json()**.
- **Tool_Buffer**: stores `SubTask_Experience` (tools/strategies) for retrieval; **add_tool()**, **sample_tool()**, **get_tool_summary()**.

All live in data_structure/experience.py.

ToDo (data structure)

Status (repo-wide): Not all ToDos are finalized. Implemented: experience/episode buffers (FIFO + add/sample), summary generation and RAG-backed memory/skill query, trainer prioritized replay, and sub-task pipeline (skill_agents). Open or partial: in/out policy scoring for buffer, per-env format helpers, completion validator, reward labeling, experience synthesis, and some trainer items (LoRA, stronger replay). RAG is not fine-tuned (frozen by design). See notes below.

1. **[Partial]** Experience buffer with in/out policy models that evaluate relative quality and relevance to the proposed latent state (intentions / sub-goals) and current state; push/pop (or add/sample) for adding/removing experiences.
   Done: Experience_Replay_Buffer and Episode_Buffer (add/sample, save/load); trainer ReplayBuffer with prioritized sampling. *Open:* policy-based in/out scoring for quality/relevance to intentions.
2. **[Partial]** Per-environment helpers to convert raw experience sequences into this format and fill blank fields; sample experiences may be required.
   Done: Experience optional fields; skill_agents uses `summary_state` / `summary` / `state`; decision_agents `get_state_summary` / `infer_intention`. *Open:* central per-env "raw → Episode" conversion helpers.
3. **[Done]** (N/A) Hooks for experience quality evaluation and prioritized replay; optional auto-ranking inside the experience buffer.
   Not required. Existing support is sufficient: ReplayBuffer has priority-weighted sampling; `Experience` has `syn_experience_quality` and a value field for replay. No plans for explicit quality hooks or auto-ranking in Episode_Buffer / Experience_Replay_Buffer.
4. **[Done]** Experience summary generation when missing; experience embedding code for RAG and experience query.
   Done: Experience.generate_summary / `generate_summary_state`; get_state_summary; EpisodicMemoryStore (RAG-backed memory query); SkillQueryEngine (RAG-backed skill query). Episode_Buffer.get_episode_summary is keyword-only (no embedding).

---

3. Skill agent

Sub-task Decomposition

This is for experience pre-processing coming from external source if unlabeled, or rollouts from sub-optimal policies.

Suppose that we have the experiences processed into the intended format, the intention of this step is to decompose the entire experience trajectories into sub-tasks and sub-trajectories, i.e., the input trajectory is unlabeled without any annotation over the entire trajectory, expected output should be sub-task labeling over the entire trajectory.

Pipeline implementation (skill_agents/)

The skill_agents module implements this via the **SkillBankAgent** pipeline: unlabeled episodes → candidate boundaries → skill-labeled segments → learned contracts (finite skill categories). Entry points: SkillBankAgent, ingest_episodes, segment_episode; see skill_agents/README.md and skill_agents/PLAN.md.

Step	Implementation	Notes
Input format	Episode with `experiences` (state/action/reward/done, etc.) and `task`. Per-step state can be `state`, `summary_state`, or `summary`; optional `sub_tasks` / `intentions` for pseudo-labels.	Experience clean-up / formulation can feed into this; intention inference (e.g. from decision_agents) can add pseudo intention over the trajectory.
Stage 1: Boundary proposal	boundary_proposal: extract signals (rule-based per env or **LLM-based** `env_name="llm"`) → propose **candidate cut points** C. Constraints: merge_radius merges nearby candidates (default 5 steps); cuts only	High-recall boundaries; no final segmentation yet.

Step	Implementation	Notes
	at C keep search O(|C|^2). Optional RAG embedder for embedding change-point scores.	
Stage 2: Sub-task labeling	infer_segmentation: decode over C with a **preference-learned scorer** (LLM provides rankings, not numeric scores). **Candidate options** = `skill_names` (existing bank IDs + `__NEW__`). Constraints: new_skill_penalty, preference_iterations, margin_threshold. Output: SegmentationResult + list of SubTask_Experience (each segment has a **skill label**).	Task types are constrained to a finite set: existing bank skills or `__NEW__`. LLM is used as a teacher for pairwise preferences; a small scorer is trained (Bradley–Terry) and DP/beam decode produces the labeling.
Stage 3: Contract learning	stage3_mvp: for each non-NEW segment, learn **effects-only contracts** (eff_add, eff_del, eff_event), verify, refine; persist to SkillBankMVP. eff_freq, min_instances_per_skill.	Converts segments into a finite, queryable skill taxonomy.
Stage 4: Bank maintenance	bank_maintenance: split low-quality skills, merge similar ones, refine contracts. materialize_new_skills promotes `__NEW__` segments meeting support/pass-rate thresholds to new named skills.	Keeps the skill set finite and consistent.

Usage (unlabeled trajectories → sub-task labeled):

```
from skill_agents import SkillBankAgent, PipelineConfig
from data_structure.experience import Episode  # experiences + task

config = PipelineConfig(
    bank_path="data/skill_bank.jsonl",
    env_name="llm+overcooked",   # or "llm" for LLM-based predicate extraction
    merge_radius=5,               # constrain boundary proximity
    new_skill_penalty=5.0,        # penalty for assigning __NEW__
    preference_iterations=3,      # active-learning rounds for preference scorer
)
agent = SkillBankAgent(config=config)
agent.load()

# Episodes from external source or sub-optimal rollouts (unlabeled)
episodes = [ep1, ep2, ...]
results = agent.ingest_episodes(episodes)   # Stage 1+2 per episode, then Stage 3
agent.run_until_stable(max_iterations=3)    # Stage 3 → Stage 4 → materialize NEW
agent.save()
# Each result: (SegmentationResult, list[SubTask_Experience]) with skill labels per segment
```

ToDo (beyond current pipeline):

1. **Experience clean-up / formulation**: Helpers to convert unformatted experiences into the Episode format (state/action/summary fields). Use intention inference (e.g. from decision_agents or a dedicated module) to add pseudo intention labels over the trajectory where missing.
2. **Tighter constraints**: Explicit max sub-task length and richer candidate options (e.g. intention-derived skill candidates per state) can be wired into Stage 1/2 config or segment filters.
3. **Training code for labeling model**: No direct supervised label signal in the current pipeline; labeling is driven by LLM preference queries and a learned scorer. For end-to-end training, one could use rollout trajectories with reward signals (sub-task assignment, task-related rewards) as in the trainer's GRPO + Hard-EM co-evolution.

4. **In-context learning**: The labeling path could be extended to generate segment labels from retrieved demonstrations (e.g. RAG over past labeled segments) instead of or in addition to the current preference-based scorer.

---

4. Decision-making agent

Reward labeling (for tasks)

Reward design for decision agent (implemented)

The decision_agents reward is implemented in reward_func.py. After every `take_action`, the **reward** tool computes a composite reward used for training-free logging and for GRPO training.

Formula: r_total = r_env + w_follow × **r_follow** + **r_cost**

Component	Description	Config / defaults
r_env	Raw environment reward from `env.step`.	From game.
r_follow	Skill-following shaping (termination-free): progress toward the active skill's **eff_add** predicates.	RewardConfig: `w_follow=0.1`, `follow_predicate_bonus=0.05`, `follow_completion_bonus=0.20`, `follow_no_progress_penalty=-0.01`.
r_cost	Costs for retrieval and skill use.	`query_mem_cost=-0.05`, `query_skill_cost=-0.05`, `call_skill_cost=-0.02`, `skill_switch_cost=-0.10`.

r_follow logic (stateful per episode): For the active skill's contract **eff_add**, the RewardComputer checks the current observation (keyword presence). It awards a per-predicate bonus for each newly satisfied predicate, a completion bonus when *all* eff_add are satisfied, and a small penalty per step with no new progress. No terminal "skill done" signal—shaping is dense.

Usage: The runner calls `RewardComputer.compute_reward(r_env, action_type, observation, active_skill_id, skill_contract)` after each step; see agent.py and run_episode_vlm_agent. Config is RewardConfig; trainer uses the same contract in trainer/decision/reward_shaping.py.

Reward labeling (possible extension, TBD)

(I am not sure if this could work, TBD)

The general idea for this part is to compute the **similarity between the next state of one experience and a target (final sub-task / task state)**, to encourage the agent to achieve maximum progress toward the goal. This would be an *additional* reward signal, separate from the implemented **r_env + r_follow + r_cost** above.

- The embedding used for this "progress toward goal" reward could be different from the one used by RAG (e.g. an LLM-produced state embedding).
- Such a function could be done with CoT to improve plausibility.

ToDo (future):

1. **Summary for states:** Use a summary generation function for the state (and optionally the target). *Already available:* get_state_summary (observation → text summary) and Experience.generate_summary / generate_summary_state for experience tuples. A future reward could consume these or a dedicated "state embedding" for similarity.
2. **Reward score (target vs next state):** Define a reward score between the **target state** and the **next state** of the current experience (e.g. after queried from a buffer). The reward score should reflect *progress toward the goal*, not raw RAG similarity: e.g. an initial state that is far in similarity but on a good trajectory toward the final state should still get high reward when it contributes to the goal.
3. **Design note:** Consider using the *change* in reward (increase/decrease) in a RAG-like state space, rather than the absolute value, to emphasize progress.

Completion validator

Verify whether the task / sub-task is completed, using the reward/completion logic above where applicable.

Point	Status	Where / note
Skill-level completion signal	**Done**	reward_func.py: when all **eff_add** predicates are satisfied in the observation, the agent receives `follow_completion_bonus`. No separate LLM "completion level" or "steps left" function.
LLM completion-level function	**Open**	Not implemented. Would load state and output completion level of current task (distinct from RAG and r_follow). e.g. high reward early → low completion score; low reward late → high completion score.
Steps-left estimator (initial + final state)	**Open**	Not implemented. Would estimate steps to reach final state; could use stored experience from RAG.

CoT and causality could be used for plausibility in future implementations.

Experience retrieval

Use RAG to retrieve experience from the buffer when generating new experience or in **training-free** mode for decision-making (in-context learning). The RAG stack is in rag/: TextEmbedder (default Qwen3-Embedding-0.6B), MemoryStore (add_texts / retrieve / rank by cosine similarity), rag/README.md.

ToDo	Status	Implementation / notes
1. Deploy RAG model to generate embeddings of experiences	**Done** (for episodic memory and skills)	EpisodicMemoryStore uses rag.retrieval.MemoryStore + `get_text_embedder()`; memories are embedded on `add()`. SkillQueryEngine embeds skill descriptions. RAG embedder is frozen during training. *Not* wired to Episode_Buffer / Experience_Replay_Buffer (they use keyword-only `get_episode_summary` / `get_experience_summary`).
2. Query experience most relevant to current state and intentions (similarity from RAG)	**Done** (for decision agent memory)	`query_memory(key)` in decision_agents calls EpisodicMemoryStore.query(key, k); key typically includes scene/objective/entities. Retrieval uses RAG cosine similarity blended with keyword overlap. See the «Do they use the RAG models?» table above.
3. Update RAG model (e.g. contrastive learning	**Done** (N/A)	RAG embedder is kept frozen (no fine-tuning). No contrastive or other RAG training is planned; retrieval uses the fixed pretrained model.

ToDo	Status	Implementation / notes
or other RAG training)		

Decision-making agent design

This part designs an agent for step-by-step decision-making in games. The framework supports both **training-free** (in-context learning with RAG) and **trainable** (reinforcement learning) modes. The implemented agent lives in **decision_agents/**.

Implemented agent: VLMDecisionAgent (decision_agents/)

The **VLM Decision Agent** plays games using a **two-turn micro-loop** per timestep: (1) **take_action** — execute exactly one environment or retrieval action; (2) **reward** — compute and log composite reward. It can optionally call **get_state_summary** or **get_intention** before take_action.

Tools

Tool	Purpose
take_action	Execute one action: either a **primitive** (move, interact, etc.) or a **retrieval action** — `QUERY_MEM(key)`, `QUERY_SKILL(key)`, `CALL_SKILL(skill_id, params)`.
reward	Compute reward for the last transition (r_env, r_follow, r_cost, r_total). Call once right after take_action.
get_state_summary	Optional: concise text summary of the observation (LLM-based for long observations).
get_intention	Optional: short phrase for current subgoal; uses last_actions, progress_notes, task.
query_skill	Used when take_action is QUERY_SKILL: retrieve procedures from the skill bank by key.
query_memory	Used when take_action is QUERY_MEM: retrieve similar past experiences (EpisodicMemoryStore).

Retrieval is **budget-limited** (e.g. every N steps or when stuck); never call query_skill and query_memory in the same timestep.

Internal state (AgentState)

The agent keeps: **current_intention**, **progress_notes**, **last_actions** (last 3–5), **stuck_counter**, **active_skill_id** / **active_skill_plan** / **skill_step_index**, **steps_without_progress**, **last_state_summary**, **last_reward**. The runner calls **update_from_tool_result()** after each tool so the next **step()** sees the updated state.

Reward (decision_agents/reward_func.py)

r_total = r_env + w_follow × r_follow + r_cost

- **r_env**: Raw environment reward from env.step.
- **r_follow**: Skill-following shaping (termination-free). The active skill's contract has **eff_add** predicates; each is satisfied if its tokens appear in the current observation. The agent gets a **per-predicate bonus**

for newly satisfied predicates, a **completion bonus** when all are satisfied, and a **no-progress penalty** when none are newly satisfied.
- **r_cost**: **query_mem_cost**, **query_skill_cost**, **call_skill_cost**, **skill_switch_cost** (when active_skill_id changes).

Defaults: `RewardConfig(w_follow=0.1, query_mem_cost=-0.05, query_skill_cost=-0.05, call_skill_cost=-0.02, skill_switch_cost=-0.10, follow_predicate_bonus=0.05, follow_completion_bonus=0.20, follow_no_progress_penalty=-0.01)`.

Helpers and integration

- **get_state_summary(observation, game, model)** — text summary for long observations.
- **infer_intention(summary, game, model, context)** — subgoal phrase.
- **EpisodicMemoryStore** — keyword-overlap memory; **add_experience()**, **query(key, k)**.
- **skill_bank_to_text(bank)**, **query_skill_bank(bank, key, top_k)** — used in prompts and by run_tool(TOOL_QUERY_SKILL).
- The agent's **skill_bank** can be a **SkillBankMVP** or a **SkillBankAgent**; **run_episode_vlm_agent(env, agent=..., max_steps=...)** runs the full two-turn loop and returns observations, actions, rewards, reward_details, done, steps, cumulative_reward.

See decision_agents/README.md for API, quick start, and reward details.

Training-free agent

This mode uses RAG to query experiences most relevant to the current situation and intentions from the experience buffer, using them as in-context learning to assist decision-making. The same **VLMDecisionAgent** is used with a **skill_bank** and optional **EpisodicMemoryStore**; no parameter updates — the backbone (e.g. GPT, Gemini, Claude) stays frozen.

Trainable agent

This mode gathers experience via interaction and synthesis and updates parameters with reinforcement learning. The **trainer/** module implements it: the VLM Decision Agent is trained with **GRPO** (retrieval as first-class actions; reward = r_env + shaping + query/call costs), and the Skill Bank is updated via **Hard-EM**. See trainer/README.md and the "Trainer Code" section below.

Inference: run and store rollouts

The **inference/** module runs the decision agent and stores rollouts in the data_structure format (`Episode` with list of `Experience`). Use this for collecting trajectories without training (e.g. for skill pipeline ingestion or replay buffers).

Run one episode and get an `Episode`:

```
from inference import run_inference, rollout_to_episode
from data_structure.experience import Episode_Buffer, Experience_Replay_Buffer

episode = run_inference(
    env,
    task="Complete level 1",
    max_steps=500,
    verbose=True,
)
```

Optional: add to buffers and/or append to a JSONL file:

```
ep_buffer = Episode_Buffer(buffer_size=100)
exp_buffer = Experience_Replay_Buffer(buffer_size=10_000)
episode = run_inference(
    env,
    task="Complete level 1",
    episode_buffer=ep_buffer,
    experience_buffer=exp_buffer,
    save_path="rollouts/episodes.jsonl",
    verbose=True,
)
```

Convert an existing `run_episode_vlm_agent` result to `Episode`:

```
from decision_agents import run_episode_vlm_agent
from inference import rollout_to_episode

rollout = run_episode_vlm_agent(env, max_steps=500)
episode = rollout_to_episode(rollout, task="My task")
# episode.experiences, episode.to_dict() for JSON
```

Storage: each episode is one JSON object per line (JSONL) when `save_path` is set. See inference/README.md for full details.

ToDo (design modules)

The decision-making agent is expected to support:

1. **Summarize the state** — GPT for training-free, Qwen (or other) for trainable.
2. **Reachable tasks / intention update** — From current state, decide whether to update sub-task or intention.
3. **(Training-free)** Integrate a RAG module to query relevant experience from the buffer.
4. **Action generation** — From current state (summary), in-episode history, and (for training-free) retrieved context, output the next action.

The counterfactual action generation module should reuse the same structure; gradients are not applied from that path.

Reward design for skill agent

This part is for the **skill agent** (skill_agents/): reward signals and training objectives used when building or evaluating the skill bank and when training experience synthesis. RAG is not fine-tuned (frozen); SFT can be used to train experience synthesis (e.g. world_model).

We adopt the idea of GDPO: normalize each kind of reward under each category. Reward categories to involve in this system include:

1. **Task completion reward** — From the environment: winning condition or survival (e.g. survive as long as possible).
2. **Sub-task completion reward** — From env or skill-level signals: e.g. occupation, kill, or skill **eff_add** satisfaction (see decision_agents/reward_func r_follow for the decision agent; skill agent may use similar notions for segment/skill quality).
3. **Format reward** — For valid outputs or adherence to action/state format.

The first two are env-related. A time-sensitive discount can be applied to penalize long episodes. For reward tied to the *decision* agent's use of skills (query_skill, call_skill), see skill_agents/tool_call_reward (r_relevance, r_utility).

---

5. Experience synthesis

Counterfactual experience generation: Given the current state and action, produce "what if" outcomes for alternative valid actions. The world_model/ module implements this via **multi-modal** (image editing) and **agentic textual** (LLM) backends, plus **experience planning** and **synthetic experience evaluation**. See world_model/Readme.md.

Implemented (**world_model/**)

Component	Description	Links
Multi-modal world model	Image editing: current frame + historical summary + instructions → synthetic next frame(s).	world_model/multi_modal/ — LongCat, Qwen-Edit, BAGEL (BAGEL_DEPLOYMENT.md)
Agentic textual world model	LLM-based: state + historical summary + instructions → next state (text), action, reward.	world_model/agentic_textual/
Experience planning	NL descriptions → step-by-step plans; optional skill-bank grounding. Plans feed into world models.	world_model/experience_planning/
Synthetic experience evaluation	LLM-as-a-judge: fidelity, consistency, instruction adherence, diversity, informativeness → verdict (ACCEPT / REFINE / REGENERATE / DISCARD).	world_model/evaluation/

Generated experience can be validated with CoT (see Completion validator); world_model/evaluation/ provides batch evaluation and verdicts.

ToDo (future)

1. **Synthesis function (richer):** Extend to (1) enumerate available actions from state; (2) optionally modify sub-task assignment under constraints; (3) infer next state and reward. Current world model already does step-level and multi-step synthesis from state + instructions.
2. **Validation:** Optional LLM-as-a-Judge for "aligned with reality" (True/False) beyond the existing evaluation dimensions.
3. **Rollout diversity:** Support single state-action and multi-step sub-task synthesis with validation in both cases; consider diversity of generated experience.
4. **Training:** SFT to improve experience synthesis quality and plausibility.

---

6. Trainer code

The training code uses experience to train agents so they (i) take better actions, (ii) get better summaries of the environment, and (iii) improve retrieval and skill-following. It lives in **trainer/** and implements co-evolution for both agents.

Implemented (trainer/)

- **Decision Agent (Agent A)** — **GRPO** (Group Relative Policy Optimization)
- **Actions**: primitives + `QUERY_MEM(key)`, `QUERY_SKILL(key)`, `CALL_SKILL(id, params)`; retrieval is first-class.
- **Reward**: r_env + w_follow × r_follow + r_cost (query/call/switch costs); optional skill-follow shaping.
- **Components**: trainer/decision/env_wrapper.py (retrieval-as-action), trainer/decision/reward_shaping.py, trainer/decision/rollout_collector.py, trainer/decision/grpo_trainer.py, trainer/decision/replay_buffer.py.

- **Launch**: `python -m trainer.decision.launch_train --config trainer/common/configs/decision_grpo.yaml`

- **SkillBank Agent (Agent B)** — **Hard-EM** (decode → update → gate)

- **Pipeline**: ingest rollouts → Stage 0 (predicates) → Stage 1 (propose cuts) → Stage 2 (decode) → Stage 3 (contracts) → Stage 4 (refine/materialize/merge/split) → SkillEval gating; no global LLM scoring.
- **Components**: trainer/skillbank/ingest_rollouts.py, trainer/skillbank/em_trainer.py, trainer/skillbank/stages/, trainer/skillbank/bank_io/ (versioned store, indices, diff logger).

- **Optional learners**: boundary classifier (trainer/skillbank/learners/boundary_trainer.py), top-2 tie-breaker (trainer/skillbank/learners/tiebreaker_trainer.py).

- **Shared**

- **Rollout schema**: trainer/common/metrics.py — `RolloutRecord` / `RolloutStep` (single source of truth for both trainers).
- **Reward contract**: trainer/decision/reward_shaping.py — `compute_reward(prev, action, next, bank_state)` → r_env, r_follow, r_cost, r_total.
- **Eval & logging**: trainer/common/eval_harness.py (fixed-seed eval, SkillBank quick eval for gating), trainer/common/logging.py, trainer/common/seeds.py.

- **Configs**: trainer/common/configs/decision_grpo.yaml, trainer/common/configs/skillbank_em.yaml.

- **Co-evolution orchestrator**: trainer/launch_coevolution.py

- Runs Decision GRPO continuously; every N episodes freezes a rollout batch, runs SkillBank EM, gates with fixed-seed eval, then commits or rolls back the bank.
- Run: `python -m trainer.launch_coevolution --decision-config trainer/common/configs/decision_grpo.yaml --skillbank-config trainer/common/configs/skillbank_em.yaml`

See trainer/README.md for full layout, milestones, metrics, and implementation order.

ToDo (future): 1. Integrate PPO / GDPO code bases and LoRA for fast fine-tuning (e.g. R1-style) where applicable. 2. Strengthen prioritized experience replay in trainer/decision/replay_buffer.py for training performance. 3. Add LoRA options to policy interface when training decision agent with parameter updates.

ToDo (unfinished / future) — consolidated

Unfinished or future work across the repo. See in-doc sections for details.

Area	Item	Status
Data structure	Experience buffer with in/out policy models for quality/relevance to intentions	**[Partial]** — buffers exist; policy-based scoring open
Data structure	Per-environment helpers to convert raw experience → Episode format	**[Partial]** — optional fields and adapters exist; central per-env helpers open
Sub-task (skill agent)	Experience clean-up / formulation; intention inference for pseudo-labels	**Open**
Sub-task (skill agent)	Tighter constraints (max sub-task length, intention-derived skill candidates)	**Open**
Sub-task (skill agent)	Training code for labeling model (supervised signal + GRPO/Hard-EM)	**Open**
Sub-task (skill agent)	In-context learning for segment labels from retrieved demonstrations	**Open**
Reward labeling	Reward score target vs next state (progress toward goal); optional state embedding	**Open** (TBD)
Completion	LLM completion-level function (distinct from r_follow)	**Open**
Completion	Steps-left estimator (initial + final state; use RAG experience)	**Open**
Experience synthesis	Richer synthesis (enumerate actions, modify sub-task under constraints)	**Open**
Experience synthesis	Optional LLM-as-Judge "aligned with reality" beyond evaluation dimensions	**Open**
Experience synthesis	Rollout diversity and SFT for synthesis quality	**Open**
Decision agent	Reachable tasks / intention update; optional RAG for training-free	**Open** (partially done: get_state_summary, infer_intention, EpisodicMemoryStore)
Trainer	PPO / GDPO / LoRA integration; stronger prioritized replay; LoRA in policy interface	**Open**